Habit Tracker Documentation

REST API Endpoints

`POST /api/auth/register`

Purpose: Regester a new account. **Authentication:** None

Headers:

```
Content-Type: application/json
```

Request:

```
{
  "name": "string",
  "email": "string",
  "password": "string"
}
```

Response:

```
{
    "token": "string",
    "user": { "Id", "string", "name":"string","email" }
}
```

`POST /api/auth/login`

Purpose: Authenticating the user. **Authentication:** None

Headers:

```
Content-Type: application/json
```

Request:

```
{
  "email": "string",
  "password": "string"
}
```

Response:

```
{
    "token": "string",
    "user": { "Id", "string", "name":"string","email" }
}
```

`GET /api/auth/me`

Purpose: Regester a new account. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Response:

```
{
    "user": { "Id", "string", "name":"string","email" }
}
```

`GET /api/habits`

Purpose: Fetch the users habits. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Response:

```
[
  {
    "userId": "string",
    "name": "string",
    "description": "string",
    "icon": "string"
  }
]
```

`POST /api/habits/add`

Purpose: Create a new habit. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Request:

```
{
  "name": "string",
  "descripion": "string",
  "Icon": "string"
}
```

Response:

```
{
  "userId": "string",
  "name": "string",
  "description": "string",
  "icon": "string"
}
```

`PATCH /api/habits/update`

Purpose: Update a habit. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Request:

```
{
  "id": "string",
  "name": "string",
  "descripion": "string",
  "Icon": "string"
}
```

Response:

```
{
  "userId": "string",
  "name": "string",
  "description": "string",
  "icon": "string"
}
```

`DELETE /api/habit/delete`

Purpose: Delete a habit. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Request:

```
{
  "habitId": "string"
}
```

Response:

```
{
  "success": "boolean"
}
```

`Patch /api/dailyEntry/:date`

Purpose: update todays daily entrys. **Authentication:** Required

Headers:

```
Content-Type: application/json
Authorization: Bearer <token>
```

Request:

```
{
  "habitId":"string",
  "date":"string",
  "completed:"boolean"
}
```

Response:

```
{
  "userId": "string",
  "date": "string",
  "habits": [
    {
      "habit": "string",
      "completed": "boolean"
    }
  ]
}
```

Dependences

Runtime Dependencies

Package	Version	licence	Purpose
bcrypt	^6.0.0	MIT	Hashing passwords
dotenv	^17.4.2	MIT	Enviroment
express	^5.2.1	MIT	webframe for node.js
jsonwebtoken	^9.0.3	MIT	An Impletation of json Web token
mongoose	^9.9.1	MIT	MongoDB ODM

DEV Dependencies

Package	Version	licence	Purpose
@tailwindcss/cli	^4.3.3	MIT	CLI for Tailwind
tailwindcss	^4.3.3	MIT	Tailwind dev tool
concurrently	^10.0.4	MIT	Run multiple commands

Credentials

An example of the credentials is in `.env.example`

```
MONGODB_URL=your-mongodb-url
JWT_SECRET=random-secret
```

Feedback

Feedback can be sent to my email address [gwainspeed@eqc-college.com]

1. Feedback is logged and catagorized and assigned a priority
2. Urgent feedback is fast-tracked.
3. Feedback is published in release/update notes.
4. Changes are reviewed and tested.
5. Feedback policy is reviewed quartely.

Guidelines

- Privacy Policy
- Terms of Service
- Copywrite

Code Commit Procedure

1. Create a new feature branch.
2. Make changes and test locally.
3. Commit changes to new branch.
4. Create a pull request to dev or main branch.
5. Review and merge changes, handle code conflicts.
6. CI/CD workflow is handled by Vercel.
7. Ensure deployment was successful by checking Vercel logs.
8. Delete feature branch.

Best practices

- Commiting frequently
- Using clear messages
- Participating in reviews
- Maintaining tests.
- Updating Documentation